

Team Number- 45

Member 1 – Vaibhav Gupta

Member 2 – Priyanshu Jha

Member 3 – Vivek Kumar Chandan

Member 4 – Abhishek Gupta

Design Pattern Findings – Reservation System

1. Builder Pattern – `Airport.java`

How We Identified It

The `Airport` class currently has **two constructors**:

```
public Airport(String name, String code, String location) { ... }  
public Airport(String name, String code, String location, String[]  
allowedAircrafts) { ... }
```

The second constructor is an extension of the first, differing only by one optional parameter (`allowedAircrafts`). This is a common signal that as the class evolves, more optional fields (e.g., `timezone`, `capacity`, `flights`) may be added, leading to **more constructors or telescoping constructor chains** — which become difficult to read and maintain.

Additionally, the class has multiple fields of the same type (`String`), which makes constructor calls ambiguous:

```
new Airport("Heathrow", "LHR", "London"); // hard to tell which arg  
is which
```

Why Builder Pattern Is a Good Fit

- **Anticipated parameter growth:** If the `Airport` class gains more optional fields, a new constructor would need to be added each time. The Builder Pattern eliminates this by allowing optional fields to be set fluently.
- **Improves readability:** Builder makes it explicit what each value represents at the call site.

- **Avoids telescoping constructors:** Instead of chaining multiple constructors, one `Builder` class handles all combinations cleanly.

Proposed Solution (Draft)

Introduce a static inner `Builder` class inside `Airport`:

```
Airport airport = new Airport.Builder("Heathrow", "LHR", "London")
    .allowedAircrafts(new String[]{"A380",
    "B747"})
    .build();
```

- Required fields (`name`, `code`, `location`) go into the `Builder` constructor.
 - Optional fields (`allowedAircrafts`, and any future fields) are set via fluent setter methods.
 - The `build()` method constructs and returns the final `Airport` object.
-

2. Factory Pattern – `PassengerPlane.java`

How We Identified It

The `PassengerPlane` constructor contains a `switch` statement that configures the object differently based on the `model` string:

```
public PassengerPlane(String model) {
    this.model = model;
    switch (model) {
        case "A380": passengerCapacity = 500; crewCapacity = 42;
        break;
        case "A350": passengerCapacity = 320; crewCapacity = 40;
        break;
        case "Embraer 190": passengerCapacity = 25; crewCapacity =
        5; break;
        case "Antonov AN2": passengerCapacity = 15; crewCapacity =
        3; break;
        default: throw new IllegalArgumentException(...);
    }
}
```

A `switch` inside a constructor that produces **varying configurations of the same class** is a strong indicator that object creation logic has grown beyond the constructor's responsibility.

Why Factory Pattern Is a Good Fit

- **Separation of concerns:** The constructor should initialize an object, not decide *which kind* of object to create. The `switch` block is creation logic, not initialization.
- **Extensibility:** Adding a new plane model currently requires modifying the constructor. With a Factory Method, a new `case` is added to one dedicated place, with no risk of breaking existing logic.
- **Encapsulation of creation logic:** The caller should not need to know the internal details of how a `PassengerPlane` is configured — the factory hides this complexity.
- **Testability:** Factory methods are easier to mock and test independently.

Proposed Solution (Draft)

Replace the public constructor with a **private constructor** and a **static factory method**:

```
// Private constructor
private PassengerPlane(String model, int passengerCapacity, int
crewCapacity) { ... }

// Factory method
public static PassengerPlane create(String model) {
    switch (model) {
        case "A380": return new PassengerPlane(model, 500, 42);
        case "A350": return new PassengerPlane(model, 320, 40);
        ...
        default: throw new IllegalArgumentException(...);
    }
}
```

Usage becomes:

```
PassengerPlane plane = PassengerPlane.create("A380");
```

For further extensibility, individual plane models could be represented as **subclasses**, each with their own fixed values.